Machine Coding: Data Table

Problem: Build a reusable Data Table: render rows from an API, sort by column, search across columns, paginate, and select rows. **Tech stack:** React (Vite, JavaScript) with plain CSS (this task folder has no Tailwind installed, so no new dependency is added) **Time limit:** 60 minutes

---

PART A: Thought process

General steps for any machine coding task

1. **Clarify** what is in scope and what is not. Write the assumptions down.
2. **List the interactions** (what the user does, what they see). This is your checklist.
3. **Sketch components** and who owns which state.
4. **Design the state**: store the minimum, derive everything else.
5. **Define data and contracts** (API shape, props of the reusable component).
6. **Trace one interaction** end to end before coding.
7. **Find the hard parts** early so they get time.
8. **Build in layers**: a working table first, then one feature at a time, each one working before the next.
9. **Cover states and edge cases**: loading, error, empty, no results, last page.
10. **Wrap up**: explain trade-offs and what you would add next.

Time budget (60 minutes)

Time	Activity
0-7 min	Requirements, clarifying questions, assumptions
7-15 min	Components, state, data contract, the filter -> sort -> paginate pipeline
15-25 min	Mock API + loading/error + plain table from a column config
25-35 min	Sorting (tri-state, types, aria-sort)
35-42 min	Search (debounced) + pagination with page reset
42-50 min	Row selection with the indeterminate header checkbox
50-55 min	Empty / no results / last page, accessibility pass, styling
55-60 min	Walk through edge cases, trade-offs, next steps

Think out loud

Say every decision and its reason as you make it: "I keep only `sort` in state and compute the sorted rows, so they can never go out of sync." The interviewer grades the reasoning as much as the result. When you take a shortcut, name it: "Client-side for now; with 100k rows I would move this to the server."

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Fetch a list of rows (users) and render them in a table.
- Columns come from a **config** (key, label, sortable, align, custom render), so the table is reusable.
- **Sort** by clicking a column header: ascending -> descending -> unsorted.
- **Search** across all columns with one input.
- **Paginate** with First / Prev / Next / Last and a "rows per page" selector.
- **Select rows** with checkboxes, plus a header checkbox for "select all on this page".
- Show "Showing 11-20 of 237 (filtered from N)".
- Show loading, error (with retry), empty and "no results" states.

Non-functional requirements

Area	What it means here
Performance	Debounce search; memoize filter and sort; render only one page of rows
Scalability	Pure filter/sort/paginate functions that can move to the server later without changing the UI
Error handling	Error message + Retry; abort the request on unmount
Accessibility	Real `<table>`, `<th scope="col">`, `aria-sort`, sort controls are buttons (keyboard), labelled checkboxes, `aria-live` summary
Reusability	`DataTable` knows nothing about users: it takes `rows`, `columns`, `rowKey`
Extensibility	Column config can grow (filter type, width, sort function) without touching the table logic

Clarifying questions

1. How many rows? Is the data loaded once (client-side) or paged by the server?
2. Single-column sort or multi-column sort (shift + click)?
3. Global search only, or per-column filters too?
4. Should selection survive sorting, searching and page changes? Is there a bulk action?
5. Do we need to keep table state in the URL (shareable, survives refresh)?
6. Any column needing custom rendering (currency, dates, badges, actions)?

Assumptions we build with

- About 250 rows, loaded once from a mocked API, processed **client-side**.
- **Single-column** sort with three states: asc, desc, none.
- **Global** case-insensitive search over the raw values of all columns, debounced 300 ms.
- Page sizes 10 / 25 / 50. Changing search, sort or page size goes back to page 1.

- Selection is stored **by row id**, so it survives sort, search and paging. The header checkbox works on the **current page**.
- Empty values (`null` , `undefined` , `''`) sort to the **end** in both directions.
- No URL sync, no column resize/reorder, no virtualization (listed in wrap-up).

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	"Loading users..." then the first 10 rows, "Page 1 of 24"
API fails	Error message with a Retry button; Retry shows loading again, then rows
Click "Age" header once	Rows sorted by age ascending, ▲ icon, `aria-sort="ascending"` , back to page 1
Click "Age" again	Descending, ▼ icon
Click "Age" a third time	Original order, neutral ↕ icon
Click "Name" while sorted by Age	Sort switches to Name ascending
Click "Email" header	Nothing: column is `sortable: false` (plain text, no button)
Type "zoya"	After 300 ms only matching rows, page 1, "(filtered from 237)"
Type something with no match	"No results for "zzzz"", header checkbox disabled, "Page 1 of 1"
Clear the search	All rows back
Next / Prev / First / Last	Moves page; Prev/First disabled on page 1, Next/Last disabled on the last page
Change rows per page to 50	50 rows, page 1, page count recalculated
Tick one row	Row highlighted, "1 selected", header checkbox becomes indeterminate
Tick header checkbox	All rows on this page selected; tick again to unselect them
Go to another page after selecting	Selection count stays, header checkbox reflects the new page
Click "Clear"	All selection removed
Keyboard: Tab to a header and press Enter	Sorts (headers are real buttons)

---

3. Design the architecture

Module	Responsibility
`App`	Fetches data, owns loading / error / success status, renders `DataTable`
`services/usersApi.js`	Mock `GET /api/users` with delay, failure flag and abort support

Module	Responsibility
`config/userColumns.js`	Column definitions for the users table (labels, render, sortable)
`components/DataTable.jsx`	Search, sort, pagination, selection state; renders the table
`components/Pagination.jsx`	Stateless page controls and rows-per-page select
`utils/table.js`	Pure functions: `filterRows`, `sortRows`, `nextSort`, `paginate`
`hooks/useDebounce.js`	Delays the search value so filtering does not run on every keystroke

```
App  (status, users, error)
 ├── usersApi.fetchUsers()        -> rows
 └── DataTable  (rows, columns from userColumns)
      ├── toolbar: search input, "N selected / Clear"
      ├── <table>
      │    ├── <thead>: select-all checkbox + sort buttons
      │    └── <tbody>: page rows | empty row
      └── footer: summary + Pagination (stateless)

utils/table.js and useDebounce are used by DataTable only.
```

Start simple: one `DataTable` component with inline header and body. `Pagination` is split out because it is a clear, stateless unit. Header and row components can be split later if they grow (resizing, row actions).

---

4. Design the state / data model

State or field	Owner	Purpose
`users`	App	Raw rows from the API
`status`	App	`'loading'`, `'success'` or `'error'`: drives which screen shows
`error`	App	Error message to display
`attempt`	App	Bumped by Retry to re-run the fetch effect
`query`	DataTable	What the user typed (controlled input)
`sort`	DataTable	`{ key, direction }`, direction is `'asc'`, `'desc'` or `null`
`page`	DataTable	Requested page (1-based)
`pageSize`	DataTable	Rows per page
`selectedIds`	DataTable	`Set` of selected row ids

```
// App
const [users, setUsers] = useState([])
const [status, setStatus] = useState('loading') // 'loading' | 'success' | 'error'
const [error, setError] = useState(null)
const [attempt, setAttempt] = useState(0)

// DataTable
```

```
const [query, setQuery] = useState('')
const [sort, setSort] = useState({ key: null, direction: null })
const [page, setPage] = useState(1)
const [pageSize, setPageSize] = useState(pageSizeOptions[0])
const [selectedIds, setSelectedIds] = useState(() => new Set())
const debouncedQuery = useDebounce(query, 300)
```

Senior note: derive, don't store

Everything below is **computed** from the state above, never stored:

Derived value	Formula
`filtered`	`filterRows(rows, columns, debouncedQuery)`
`sorted`	`sortRows(filtered, sort)`
`totalPages`	`max(1, ceil(sorted.length / pageSize))`
`currentPage`	`min(page, totalPages)` (clamped, so it can never point past the end)
`pageRows`	`paginate(sorted, currentPage, pageSize)`
`allOnPageSelected`, `someOnPageSelected`	From `pageRows` ids and `selectedIds`
"Showing X-Y of Z"	From `currentPage`, `pageSize`, `sorted.length`

If you stored `sortedRows` or `filteredRows` in state you would need effects to keep them in sync, and every sync is a chance for a stale bug. **Ownership:** data fetching lives in `App` (the table does not care where rows come from); view state (search, sort, page, selection) lives in `DataTable` because nothing else needs it. If a parent ever needs it (URL sync, bulk delete), lift it up and make `DataTable` controlled.

---

5. Design the data structures and contracts

Example row

```
{
  id: 17,                    // stable unique id: React key + selection key
  name: 'Ishaan Patel',
  email: 'ishaan.patel17@example.com',
  department: 'Sales',
  age: 29,                   // number: sorts numerically
  salary: 85000,             // number in data, formatted only when rendered
  joinedAt: '2019-06-17',    // ISO date string: sorts correctly as text
}
```

Why a stable `id`: the row index changes whenever you sort or filter. Using the index as React key or as the selection key would select the wrong row after sorting. The id never changes.

Why raw values: keep numbers as numbers and dates as ISO strings in the data. Formatting (`₹85,000`) happens in the column `render`, so sorting works on real values.

Column config contract

```
{
  key: 'salary',             // field name in the row (required, unique)
  label: 'Salary',           // header text (required)
  sortable: true,            // default true; false = no sort button
  align: 'right',            // optional class for numbers
  render: (value, row) => node, // optional custom cell rendering
}
```

`DataTable` props

Prop	Type	Default	Meaning
`rows`	`object[]`	required	Data to show
`columns`	`Column[]`	required	Column config
`rowKey`	`string`	`'id'`	Field used as React key and selection key
`caption`	`string`	-	Accessible table name
`pageSizeOptions`	`number[]`	`[10, 25, 50]`	Page size choices, first one is the default

API contract (mocked)

```
GET /api/users
200 -> [{ id, name, email, department, age, salary, joinedAt }, ...]
500 -> { message: "Could not load users. Please try again." }
```

```
fetchUsers({ signal, fail }) // Promise<User[]>
// resolves after 600 ms, rejects with Error when fail = true,
// rejects with AbortError when signal aborts
```

For a server-side version the contract would become `GET /api/users?page=2&pageSize=25&sort=age&order=desc&q=zoya -> { rows, total }`.

---

6. Trace the data flow

Interaction: user clicks the "Age" header while on page 3.

```
Click <button> in the Age <th>
        |
        ▼
handleSort('age')
  setSort(prev => nextSort(prev, 'age'))   // {null,null} -> {age, asc}
  setPage(1)                               // a new order makes page 3 meaningless
```

```
        |  (both updates batched into one render)
        ▼
Render
  filtered  = memo hit (rows, query unchanged)
  sorted    = sortRows(filtered, {age, asc})   // copy + stable sort
  totalPages, currentPage = 1
  pageRows  = sorted.slice(0, 10)
        |
        ▼
Output
  <th aria-sort="ascending"> with ▲, youngest 10 rows,
  "Showing 1-10 of 237", "Page 1 of 24",
  header checkbox recomputed for the new page ids (selection kept by id)
```

The search flow is the same with one extra hop: keystroke -> `query` (input updates instantly) + `setPage(1)` -> 300 ms later `debouncedQuery` -> `filtered` recomputes -> `sorted` -> `pageRows`.

---

7. Important technical challenges

Challenge 1: The pipeline order and keeping the page valid

Problem: You are on page 20 of 24 and type "zoya". Only 21 rows match (3 pages). If `page` stays 20 you render an empty table even though results exist. Also, if you paginate before sorting, you sort only the 10 visible rows, which is wrong.

Solution: Always derive in this order: **filter -> sort -> paginate**. Reset to page 1 inside the handlers that change the result set (search, sort, page size). As a safety net, **clamp** the page while rendering instead of syncing it with an effect.

```
const filtered = useMemo(() => filterRows(rows, columns, debouncedQuery), [rows, columns,
debouncedQuery])
const sorted = useMemo(() => sortRows(filtered, sort), [filtered, sort])
const totalPages = Math.max(1, Math.ceil(sorted.length / pageSize))
const currentPage = Math.min(page, totalPages) // clamp instead of syncing with an effect
const pageRows = useMemo(() => paginate(sorted, currentPage, pageSize), [sorted, currentPage,
pageSize])

function handleSearch(e) {
  setQuery(e.target.value)
  setPage(1)
}
```

Why not `useEffect(() => setPage(1), [query])`? It renders once with the wrong page, then again. Doing it in the event handler is one render and easier to follow.

Challenge 2: Sorting correctly (types, empty values, stability, no mutation)

Problem: A naive `rows.sort((a, b) => a[key] > b[key] ? 1 : -1)` has four bugs:

- It **mutates** the array from props/state (React will not see a change, and the "unsorted" order is lost).
- `"item 10"` sorts before `"item 2"`, and `"apple"` vs `"Banana"` depends on case.

- `null` ages end up in random places, and flipping direction moves them to the top.
- Equal values jump around between clicks if the comparator is inconsistent.

Solution: Copy before sorting, compare numbers with subtraction and strings with `Intl.Collator` (numeric, case-insensitive), push empty values last regardless of direction, and rely on `Array.prototype.sort` being stable (ES2019) so ties keep their original order. Sort direction cycles none -> asc -> desc -> none.

```
const collator = new Intl.Collator(undefined, { numeric: true, sensitivity: 'base' })

export function sortRows(rows, { key, direction }) {
  if (!key || !direction) return rows
  const dir = direction === 'asc' ? 1 : -1
  return [...rows].sort((r1, r2) => {
    const a = r1[key]
    const b = r2[key]
    const aEmpty = a == null || a === ''
    const bEmpty = b == null || b === ''
    if (aEmpty || bEmpty) return aEmpty - bEmpty // empty values always last
    return compareValues(a, b) * dir
  })
}

export function nextSort(sort, key) {
  if (sort.key !== key) return { key, direction: 'asc' }
  if (sort.direction === 'asc') return { key, direction: 'desc' }
  return { key: null, direction: null }
}
```

Challenge 3: Selection that survives sort, search and paging

Problem: You tick row 3 on page 1, then sort by age. If selection was stored by index (`[0, 2]`), a different person is now "selected". The header checkbox also has three visual states: none, some, all of this page. HTML has no `indeterminate` attribute; it is a DOM property only.

Solution: Store a `Set` of **row ids**. Always create a new `Set` when updating (immutability, so React re-renders). Compute the header state from the current page ids, and set `indeterminate` through a callback ref.

```
const pageIds = pageRows.map((row) => row[rowKey])
const allOnPageSelected = pageIds.length > 0 && pageIds.every((id) => selectedIds.has(id))
const someOnPageSelected = pageIds.some((id) => selectedIds.has(id))

function togglePage() {
  setSelectedIds((prev) => {
    const next = new Set(prev)
    for (const id of pageIds) {
      if (allOnPageSelected) next.delete(id)
      else next.add(id)
    }
    return next
  })
}

<input
  type="checkbox"
  checked={allOnPageSelected}
```

```
  ref={(el) => { if (el) el.indeterminate = someOnPageSelected && !allOnPageSelected }}
  onChange={togglePage}
/>
```

Challenge 4: Search that stays fast, and a fetch that cannot leak

Problem: Filtering 10k rows on every keystroke makes typing laggy. Separately, if the component unmounts (or StrictMode mounts twice) while the request is in flight, a late response calls `setState` on a dead component or overwrites newer data.

Solution: Keep the input controlled by `query` (it feels instant) but filter on `useDebounce(query, 300)`; memoize each pipeline stage. For the fetch, pass an `AbortController` signal and abort in the effect cleanup; ignore `AbortError`.

```
export function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value)
  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay)
    return () => clearTimeout(timer)
  }, [value, delay])
  return debounced
}

useEffect(() => {
  const controller = new AbortController()
  fetchUsers({ signal: controller.signal })
    .then((data) => { setUsers(data); setStatus('success') })
    .catch((err) => {
      if (err.name === 'AbortError') return
      setError(err.message)
      setStatus('error')
    })
  return () => controller.abort()
}, [attempt])
```

---

8. Implement in the correct order

#	Step	Working result after the step
1	Mock `fetchUsers` with deterministic data, delay and abort	Data available
2	`App` with loading / error / success + Retry	All three screens visible
3	`userColumns` config + `DataTable` rendering all rows	A plain table
4	Pure `paginate` + `Pagination` component (page, page size)	10 rows per page, navigation works
5	Pure `sortRows` + `nextSort`, header buttons with icons	Click to sort, tri-state
6	`aria-sort`, `scope="col"`, right-aligned numbers	Accessible headers
7	`filterRows` + search input + `useDebounce`	Search works

#	Step	Working result after the step
8	Reset page in handlers + clamp `currentPage`	Never stuck on an empty page
9	Selection `Set` by id, row checkbox, header checkbox with indeterminate	Selection works across pages
10	Empty / no results row, "Showing X-Y of Z", `aria-live`	All states covered
11	Styling: sticky header, horizontal scroll on small screens, selected row highlight	Polished
12	Node assert check of the pure functions	Logic verified

---

9. Cover all states and edge cases

State / edge case	What the user sees / what the code does
Initial / loading	"Loading users..." with `role="status"`
Error	Message with `role="alert"` and a Retry button
Retry	Back to loading, then success
Success	First page of rows
API returns `[]`	"No data to show", "Showing 0-0 of 0", "Page 1 of 1", header checkbox disabled
Search with no match	"No results for "...""
Search with spaces / caps	Trimmed and lower-cased before matching
Last page partially filled	Shows 7 rows, "Showing 231-237 of 237", Next/Last disabled
Page beyond range (data shrank)	`currentPage` clamped to `totalPages`
Page size bigger than data	One page, all rows
Sorting a non-sortable column	No button rendered
`null` / empty values	Shown as "—", always sorted last
Equal values	Stable order (original order kept)
Mixed-case strings, "item 2" vs "item 10"	`Intl.Collator` with `numeric` and `sensitivity: 'base'`
Selection then sort / search / page change	Kept, because it is stored by id
Some rows on page selected	Header checkbox indeterminate
Unmount during fetch / StrictMode double effect	Request aborted, no state update
Rapid typing	Only one filter run, 300 ms after the last keystroke
Narrow screen	Table scrolls horizontally inside its wrapper; header stays sticky

---

10. Final structure

```
_35_data_table/
├── index.html
├── package.json
└── src/
    ├── main.jsx                # entry (unchanged)
    ├── App.jsx                 # fetch + loading/error/success
    ├── App.css                 # table, toolbar, pagination styles
    ├── index.css               # base styles
    ├── config/
    │   └── userColumns.js      # column definitions for users
    ├── services/
    │   └── usersApi.js         # mock GET /api/users
    ├── utils/
    │   └── table.js            # filterRows, sortRows, nextSort, paginate
    ├── hooks/
    │   └── useDebounce.js
    └── components/
        ├── DataTable.jsx       # reusable table with search/sort/page/select
        └── Pagination.jsx      # stateless page controls
```

11. Full working code

`src/services/usersApi.js`

```
const FIRST = ['Aarav', 'Diya', 'Kabir', 'Meera', 'Rohan', 'Sara', 'Vihaan', 'Anaya', 'Ishaan',
'Zoya', 'Arjun']
const LAST = ['Sharma', 'Verma', 'Iyer', 'Khan', 'Patel', 'Reddy', 'Gupta', 'Das', 'Nair']
const DEPARTMENTS = ['Engineering', 'Design', 'Sales', 'Marketing', 'HR']

// Deterministic mock data: same rows on every load, so sorting/paging is easy to verify.
const USERS = Array.from({ length: 237 }, (_, i) => {
  const first = FIRST[i % FIRST.length]
  const last = LAST[(i * 7) % LAST.length]
  return {
    id: i + 1,
    name: `${first} ${last}`,
    email: `${first}.${last}${i + 1}@example.com`.toLowerCase(),
    department: DEPARTMENTS[(i * 3) % DEPARTMENTS.length],
    age: 21 + ((i * 13) % 40),
    salary: 30000 + ((i * 7919) % 120) * 1000,
    joinedAt: new Date(Date.UTC(2016 + (i % 9), (i * 5) % 12, (i % 28) + 1)).toISOString().slice(0,
10),
  }
})

// Mock GET /api/users -> User[]. Rejects with AbortError when the signal aborts.
export function fetchUsers({ signal, fail = false } = {}) {
  return new Promise((resolve, reject) => {
    const timer = setTimeout(() => {
      if (fail) reject(new Error('Could not load users. Please try again.'))
      else resolve(USERS)
```

```
    }, 600)
    signal?.addEventListener('abort', () => {
      clearTimeout(timer)
      reject(new DOMException('Request aborted', 'AbortError'))
    })
  })
}
```

`src/config/userColumns.js`

```
const currency = new Intl.NumberFormat('en-IN', { style: 'currency', currency: 'INR',
maximumFractionDigits: 0 })

// Column config: the table renders whatever this array describes.
export const userColumns = [
  { key: 'id', label: 'ID', align: 'right' },
  { key: 'name', label: 'Name' },
  { key: 'email', label: 'Email', sortable: false },
  { key: 'department', label: 'Department' },
  { key: 'age', label: 'Age', align: 'right' },
  { key: 'salary', label: 'Salary', align: 'right', render: (value) => currency.format(value) },
  { key: 'joinedAt', label: 'Joined' },
]
```

`src/utils/table.js`

```
const collator = new Intl.Collator(undefined, { numeric: true, sensitivity: 'base' })

export function filterRows(rows, columns, query) {
  const q = query.trim().toLowerCase()
  if (!q) return rows
  return rows.filter((row) =>
    columns.some((col) => String(row[col.key] ?? '').toLowerCase().includes(q)),
  )
}

export function compareValues(a, b) {
  if (typeof a === 'number' && typeof b === 'number') return a - b
  // numeric collation: "item 2" < "item 10", case-insensitive; ISO dates sort correctly as strings
  return collator.compare(String(a), String(b))
}

export function sortRows(rows, { key, direction }) {
  if (!key || !direction) return rows
  const dir = direction === 'asc' ? 1 : -1
  // copy: never mutate the source rows; Array.prototype.sort is stable, so ties keep their order
  return [...rows].sort((r1, r2) => {
    const a = r1[key]
    const b = r2[key]
    const aEmpty = a == null || a === ''
    const bEmpty = b == null || b === ''
    if (aEmpty || bEmpty) return aEmpty - bEmpty // empty values always last, in both directions
    return compareValues(a, b) * dir
  })
}
```

```
// none -> asc -> desc -> none; a different column starts at asc
export function nextSort(sort, key) {
  if (sort.key !== key) return { key, direction: 'asc' }
  if (sort.direction === 'asc') return { key, direction: 'desc' }
  return { key: null, direction: null }
}

export function paginate(rows, page, pageSize) {
  const start = (page - 1) * pageSize
  return rows.slice(start, start + pageSize)
}
```

`src/hooks/useDebounce.js`

```
import { useEffect, useState } from 'react'

export function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value)
  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay)
    return () => clearTimeout(timer)
  }, [value, delay])
  return debounced
}
```

`src/components/Pagination.jsx`

```
export default function Pagination({ page, totalPages, pageSize, pageSizeOptions, onPageChange,
onPageSizeChange }) {
  return (
    <nav className="pagination" aria-label="Table pagination">
      <label>
        Rows per page{' '}
        <select value={pageSize} onChange={(e) => onPageSizeChange(Number(e.target.value))}>
          {pageSizeOptions.map((size) => (
            <option key={size} value={size}>{size}</option>
          ))}
        </select>
      </label>
      <div className="pager">
        <button type="button" onClick={() => onPageChange(1)} disabled={page === 1} aria-label="First
page">«</button>
        <button type="button" onClick={() => onPageChange(page - 1)} disabled={page === 1} aria-
label="Previous page">‹</button>
        <span>Page {page} of {totalPages}</span>
        <button type="button" onClick={() => onPageChange(page + 1)} disabled={page === totalPages}
aria-label="Next page">›</button>
        <button type="button" onClick={() => onPageChange(totalPages)} disabled={page === totalPages}
aria-label="Last page">»</button>
      </div>
    </nav>
  )
}
```

`src/components/DataTable.jsx`

```
import { useMemo, useState } from 'react'
import { useDebounce } from '../hooks/useDebounce'
import { filterRows, nextSort, paginate, sortRows } from '../utils/table'
import Pagination from './Pagination'

const SORT_ICON = { asc: '▲', desc: '▼' }

export default function DataTable({ rows, columns, rowKey = 'id', caption, pageSizeOptions = [10, 25,
50] }) {
  const [query, setQuery] = useState('')
  const [sort, setSort] = useState({ key: null, direction: null })
  const [page, setPage] = useState(1)
  const [pageSize, setPageSize] = useState(pageSizeOptions[0])
  const [selectedIds, setSelectedIds] = useState(() => new Set())
  const debouncedQuery = useDebounce(query, 300)

  // pipeline: rows -> filter -> sort -> paginate (all derived, nothing duplicated in state)
  const filtered = useMemo(() => filterRows(rows, columns, debouncedQuery), [rows, columns,
debouncedQuery])
  const sorted = useMemo(() => sortRows(filtered, sort), [filtered, sort])
  const totalPages = Math.max(1, Math.ceil(sorted.length / pageSize))
  const currentPage = Math.min(page, totalPages) // clamp instead of syncing with an effect
  const pageRows = useMemo(() => paginate(sorted, currentPage, pageSize), [sorted, currentPage,
pageSize])

  const pageIds = pageRows.map((row) => row[rowKey])
  const allOnPageSelected = pageIds.length > 0 && pageIds.every((id) => selectedIds.has(id))
  const someOnPageSelected = pageIds.some((id) => selectedIds.has(id))
  const from = sorted.length === 0 ? 0 : (currentPage - 1) * pageSize + 1
  const to = Math.min(currentPage * pageSize, sorted.length)

  function handleSearch(e) {
    setQuery(e.target.value)
    setPage(1)
  }

  function handleSort(key) {
    setSort((prev) => nextSort(prev, key))
    setPage(1)
  }

  function handlePageSize(size) {
    setPageSize(size)
    setPage(1)
  }

  function toggleRow(id) {
    setSelectedIds((prev) => {
      const next = new Set(prev)
      if (next.has(id)) next.delete(id)
      else next.add(id)
      return next
    })
  }

  function togglePage() {
    setSelectedIds((prev) => {
```

```
      const next = new Set(prev)
      for (const id of pageIds) {
        if (allOnPageSelected) next.delete(id)
        else next.add(id)
      }
      return next
    })
  }

  function ariaSort(key) {
    if (sort.key !== key) return 'none'
    return sort.direction === 'asc' ? 'ascending' : 'descending'
  }

  return (
    <div className="data-table">
      <div className="toolbar">
        <input
          type="search"
          value={query}
          onChange={handleSearch}
          placeholder="Search all columns…"
          aria-label="Search table"
        />
        {selectedIds.size > 0 && (
          <span className="selection">
            {selectedIds.size} selected{' '}
            <button type="button" className="link" onClick={() => setSelectedIds(new
Set())}>Clear</button>
          </span>
        )}
      </div>

      <div className="table-scroll">
        <table>
          {caption && <caption>{caption}</caption>}
          <thead>
            <tr>
              <th scope="col" className="check">
                <input
                  type="checkbox"
                  aria-label="Select all rows on this page"
                  checked={allOnPageSelected}
                  ref={(el) => {
                    if (el) el.indeterminate = someOnPageSelected && !allOnPageSelected
                  }}
                  onChange={togglePage}
                  disabled={pageIds.length === 0}
                />
              </th>
              {columns.map((col) => (
                <th key={col.key} scope="col" aria-sort={col.sortable === false ? undefined :
ariaSort(col.key)} className={col.align}>
                  {col.sortable === false ? (
                    col.label
                  ) : (
                    <button type="button" className="sort-btn" onClick={() => handleSort(col.key)}>
                      {col.label}
                      <span aria-hidden="true" className="sort-icon">
                        {sort.key === col.key ? SORT_ICON[sort.direction] : '↕'}
```

```
                      </span>
                    </button>
                  )}
                </th>
              ))}
            </tr>
          </thead>
          <tbody>
            {pageRows.length === 0 ? (
              <tr>
                <td colSpan={columns.length + 1} className="empty">
                  {debouncedQuery ? `No results for "${debouncedQuery}"` : 'No data to show'}
                </td>
              </tr>
            ) : (
              pageRows.map((row) => {
                const id = row[rowKey]
                const selected = selectedIds.has(id)
                return (
                  <tr key={id} aria-selected={selected} className={selected ? 'selected' : undefined}>
                    <td className="check">
                      <input type="checkbox" aria-label={`Select row ${id}`} checked={selected}
onChange={() => toggleRow(id)} />
                    </td>
                    {columns.map((col) => (
                      <td key={col.key} className={col.align}>
                        {col.render ? col.render(row[col.key], row) : (row[col.key] ?? '—')}
                      </td>
                    ))}
                  </tr>
                )
              })
            )}
          </tbody>
        </table>
      </div>

      <div className="footer">
        <p aria-live="polite" className="summary">
          Showing {from}–{to} of {sorted.length}
          {sorted.length !== rows.length && ` (filtered from ${rows.length})`}
        </p>
        <Pagination
          page={currentPage}
          totalPages={totalPages}
          pageSize={pageSize}
          pageSizeOptions={pageSizeOptions}
          onPageChange={setPage}
          onPageSizeChange={handlePageSize}
        />
      </div>
    </div>
  )
}
```

`src/App.jsx`

```
import { useEffect, useState } from 'react'
import DataTable from './components/DataTable'
import { userColumns } from './config/userColumns'
import { fetchUsers } from './services/usersApi'
import './App.css'

// open the app with ?fail to see the error state (the retry then succeeds)
const SIMULATE_FAILURE = new URLSearchParams(window.location.search).has('fail')

export default function App() {
  const [users, setUsers] = useState([])
  const [status, setStatus] = useState('loading') // 'loading' | 'success' | 'error'
  const [error, setError] = useState(null)
  const [attempt, setAttempt] = useState(0)

  useEffect(() => {
    const controller = new AbortController()
    fetchUsers({ signal: controller.signal, fail: SIMULATE_FAILURE && attempt === 0 })
      .then((data) => {
        setUsers(data)
        setStatus('success')
      })
      .catch((err) => {
        if (err.name === 'AbortError') return
        setError(err.message)
        setStatus('error')
      })
    return () => controller.abort()
  }, [attempt])

  function retry() {
    setStatus('loading')
    setAttempt((n) => n + 1)
  }

  return (
    <main className="app">
      <h1>Employees</h1>
      {status === 'loading' && <p role="status" className="state">Loading users…</p>}
      {status === 'error' && (
        <div role="alert" className="state error">
          <p>{error}</p>
          <button type="button" onClick={retry}>Retry</button>
        </div>
      )}
      {status === 'success' && <DataTable rows={users} columns={userColumns} caption="Company
employees" />}
    </main>
  )
}
```

`src/index.css`

```
:root {
  font-family: system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
```

```
  line-height: 1.5;
  color: #1f2937;
  background: #f3f4f6;
}

* { box-sizing: border-box; }

body { margin: 0; }

button, input, select { font: inherit; }
```

`src/App.css`

```
.app { max-width: 1100px; margin: 0 auto; padding: 24px 16px; }
.app h1 { margin: 0 0 16px; font-size: 1.5rem; }

.state { padding: 32px; text-align: center; background: #fff; border-radius: 8px; }
.state.error { color: #b91c1c; }
.state button { padding: 6px 16px; border: 1px solid #b91c1c; border-radius: 6px; background: #fff;
color: #b91c1c; cursor: pointer; }

.data-table { background: #fff; border-radius: 8px; box-shadow: 0 1px 3px rgb(0 0 0 / 0.1); }

.toolbar { display: flex; gap: 16px; align-items: center; justify-content: space-between; padding:
12px; flex-wrap: wrap; }
.toolbar input { flex: 1; min-width: 200px; max-width: 360px; padding: 8px 12px; border: 1px solid
#d1d5db; border-radius: 6px; }
.selection { font-size: 0.9rem; color: #4b5563; }
.link { border: 0; background: none; color: #2563eb; cursor: pointer; padding: 0; text-decoration:
underline; }

/* horizontal scroll on small screens instead of squeezing columns */
.table-scroll { overflow: auto; max-height: 70vh; border-top: 1px solid #e5e7eb; }
table { width: 100%; border-collapse: collapse; font-size: 0.9rem; }
caption { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0); }
th, td { padding: 10px 12px; text-align: left; white-space: nowrap; border-bottom: 1px solid #e5e7eb;
}
th { position: sticky; top: 0; z-index: 1; background: #f9fafb; font-weight: 600; }
th.right, td.right { text-align: right; }
th.check, td.check { width: 40px; }
tbody tr:hover { background: #f9fafb; }
tbody tr.selected { background: #eff6ff; }
td.empty { text-align: center; padding: 40px; color: #6b7280; }

.sort-btn { display: inline-flex; gap: 6px; align-items: center; border: 0; background: none; padding:
0; font-weight: 600; cursor: pointer; color: inherit; }
.sort-icon { font-size: 0.7rem; color: #9ca3af; }
th[aria-sort='ascending'] .sort-icon, th[aria-sort='descending'] .sort-icon { color: #2563eb; }
.sort-btn:focus-visible, .pager button:focus-visible { outline: 2px solid #2563eb; outline-offset:
2px; }

.footer { display: flex; gap: 12px; align-items: center; justify-content: space-between; padding:
12px; flex-wrap: wrap; }
.summary { margin: 0; font-size: 0.9rem; color: #4b5563; }
.pagination { display: flex; gap: 16px; align-items: center; flex-wrap: wrap; font-size: 0.9rem; }
.pagination select { padding: 4px; border-radius: 4px; border: 1px solid #d1d5db; }
.pager { display: flex; gap: 4px; align-items: center; }
.pager span { padding: 0 8px; }
```

```
.pager button { min-width: 32px; padding: 4px 8px; border: 1px solid #d1d5db; border-radius: 4px;
background: #fff; cursor: pointer; }
.pager button:disabled { opacity: 0.4; cursor: not-allowed; }
```

`src/main.jsx` (unchanged from the Vite template)

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Quick logic check (`node check.mjs`)

```
import assert from 'node:assert/strict'
import { filterRows, sortRows, nextSort, paginate } from './src/utils/table.js'
import { fetchUsers } from './src/services/usersApi.js'

const cols = [{ key: 'name' }, { key: 'age' }]
const rows = [
  { id: 1, name: 'item 10', age: 30 },
  { id: 2, name: 'Item 2', age: null },
  { id: 3, name: 'apple', age: 25 },
  { id: 4, name: 'Banana', age: 25 },
]
assert.deepEqual(filterRows(rows, cols, '  ITEM ').map((r) => r.id), [1, 2])
assert.deepEqual(filterRows(rows, cols, '25').map((r) => r.id), [3, 4])
assert.equal(filterRows(rows, cols, ''), rows)
assert.deepEqual(sortRows(rows, { key: 'name', direction: 'asc' }).map((r) => r.id), [3, 4, 2, 1])
assert.deepEqual(sortRows(rows, { key: 'age', direction: 'asc' }).map((r) => r.id), [3, 4, 1, 2])
assert.deepEqual(sortRows(rows, { key: 'age', direction: 'desc' }).map((r) => r.id), [1, 3, 4, 2]) //
stable + empty last
assert.equal(sortRows(rows, { key: null, direction: null }), rows)
assert.deepEqual(rows.map((r) => r.id), [1, 2, 3, 4]) // source not mutated
let s = { key: null, direction: null }
s = nextSort(s, 'age'); assert.deepEqual(s, { key: 'age', direction: 'asc' })
s = nextSort(s, 'age'); assert.deepEqual(s, { key: 'age', direction: 'desc' })
s = nextSort(s, 'age'); assert.deepEqual(s, { key: null, direction: null })
assert.deepEqual(nextSort({ key: 'age', direction: 'desc' }, 'name'), { key: 'name', direction: 'asc'
})
assert.deepEqual(paginate([1, 2, 3, 4, 5], 2, 2), [3, 4])
assert.deepEqual(paginate([1, 2, 3, 4, 5], 3, 2), [5])
const users = await fetchUsers()
assert.equal(users.length, 237)
assert.equal(new Set(users.map((u) => u.id)).size, 237)
const c = new AbortController()
const p = fetchUsers({ signal: c.signal }); c.abort()
await assert.rejects(p, { name: 'AbortError' })
await assert.rejects(fetchUsers({ fail: true }), /Could not load/)
console.log('all checks passed')
```

12. Wrap-up

Edge cases handled

- Loading, error with retry, empty data, no search results, partial last page.
- Page clamped when the result set shrinks; page reset on search, sort and page size change.
- Numeric vs text sorting, case-insensitive natural order, empty values last, stable ties, no mutation.
- Selection by id that survives sort, search and paging, with an indeterminate header checkbox.
- Request aborted on unmount; rapid typing debounced.
- Keyboard-operable sort buttons, `aria-sort`, labelled checkboxes, live summary.

Trade-offs to mention

Decision	Why	Cost
Client-side processing	Simple, instant sorting/search for a few thousand rows	Does not scale to 100k+ rows or huge payloads
Derived pipeline with `useMemo`	No sync bugs, easy to reason about	Recomputes the whole sort when sort changes (fine at this size)
Uncontrolled view state inside `DataTable`	Easy to drop into any page	Parent cannot read sort/page (lift it up when needed)
Search on raw values	Simple and fast	Searching "₹85,000" does not match the raw `85000`
Header checkbox = current page only	Clear and predictable	"Select all 237" needs an extra banner (like Gmail)
Plain CSS, no table library	No dependency, shows fundamentals	TanStack Table would give resizing, grouping, etc. for free

What to add with more time

- **Tests:** React Testing Library for sort clicks, search, paging, selection; the pure functions already have asserts.
- **Server-side mode:** send `page, pageSize, sort, order, q` to the API, show `total` from the response, keep the same UI.
- **Virtualization** (windowing) when one page must show thousands of rows.
- **URL sync** of `q`, `sort`, `page` with `URLSearchParams` so state survives refresh and can be shared.
- Per-column filters, multi-column sort (shift + click), column resize / reorder / hide.
- "Select all N matching rows" banner and bulk actions.
- Skeleton rows instead of the loading text, and sticky first column on mobile.

13. Counter questions the interviewer may ask

Requirements

Q1. What if the data is 1 million rows?

You cannot ship 1M rows to the browser. Move filter, sort and paginate to the server: `GET /users?page=3&pageSize=25&sort=age&order=desc&q=zoya -> { rows, total }`. The UI stays the same; only where the pipeline runs changes. Debounce the search request and cancel stale ones with `AbortController`.

Q2. Why client-side processing at all?

For a few hundred or few thousand rows, one request plus in-memory sorting is faster for the user (no round trip per click) and much simpler. The pure `utils/table.js` functions make the switch to server-side a contained change.

Q3. How would you support multi-column sort?

Store `sort` as an array: `[{ key: 'department', direction: 'asc' }, { key: 'age', direction: 'desc' }]`. Shift + click appends or cycles a key. The comparator walks the array and returns the first non-zero comparison.

State design

Q4. Why not store `filteredRows` and `sortedRows` in state?

They are fully determined by `rows`, `query` and `sort`. Storing them means effects to keep them in sync, an extra render each time, and the risk that one is updated and the other is not. Derive and memoize instead.

Q5. Why is selection a `Set` of ids and not an array of indexes?

Indexes change when you sort, filter or change page, so the wrong rows would be selected. Ids are stable. A `Set` gives O(1) `has` checks per row. Always create a new `Set` when updating so React sees a new reference.

Q6. Why does `DataTable` own its state instead of the parent?

Nothing else needs it today, so keeping it local makes the component easy to drop in. If the parent needs it (URL sync, bulk delete, server-side fetching), make it controlled: accept `sort`, `onSortChange`, `page`, `onPageChange` props, like an input's `value` / `onChange`.

Challenge 1: pipeline and page

Q7. Why reset the page in handlers instead of a `useEffect` on `query`?

An effect runs after render, so the user briefly sees the wrong page and React renders twice. Calling `setPage(1)` in the same handler batches both updates into one render. The clamp `Math.min(page, totalPages)` covers any case the handlers miss (for example, new data arriving).

Q8. Why filter before sort?

Sorting is O(n log n). Filtering first shrinks n, so you sort fewer rows. The result is the same either way, but paginate must always be last.

Challenge 2: sorting

Q9. Why not just `rows.sort(...)`?

`sort` mutates in place. `rows` comes from props/state; mutating it breaks React's change detection and loses the original order needed for the "unsorted" state. Use `[...rows].sort(...)` (or `toSorted` in modern browsers).

Q10. What does `Intl.Collator` give you over `localeCompare` or `>`?

Natural numeric order ("item 2" before "item 10"), case and accent insensitivity, and correct ordering for other languages. Creating one collator and reusing it is faster than calling `localeCompare` with options every time.

Q11. How would you sort a column whose display differs from its value (e.g. status labels)?

Add an optional `sortValue: (row) => any` (or a custom `compare`) to the column config and use it in `sortRows` when present. The table logic stays generic.

Challenge 3: selection

Q12. Why is `indeterminate` set with a ref?

It is a DOM property, not an HTML attribute, so JSX cannot set it. A callback ref runs after each render and sets `el.indeterminate` from the derived state.

Q13. How would you add "Select all 237 rows"?

When the whole page is selected, show a banner "All 10 on this page selected. Select all 237?". Store a flag `allMatchingSelected` (plus excluded ids) instead of putting 237 ids in the set; in server mode you send the filter to the bulk API rather than ids.

Challenge 4: search and fetch

Q14. Debounce or throttle for search?

Debounce: we want to run once after the user stops typing. Throttle runs at a fixed rate while typing, which wastes work on half-typed words. For an API-backed search, debounce plus aborting the previous request.

Q15. What if the API is slow or responses come back out of order?

Show the loading state (or keep old rows with a small spinner), abort the previous request when a new one starts, and ignore `AbortError`. Without abort, a slow response for "zo" can arrive after "zoya" and overwrite the newer results.

Q16. Why does the effect abort on cleanup? Isn't the mock instant?

In development StrictMode mounts, unmounts and remounts, so the effect runs twice. Aborting the first request prevents a double `setState`. In production it protects against unmounting while the request is in flight.

Implementation steps

Q17. Why is the sort control a `<button>` inside `<th>` and not `onClick` on the `<th>`?

A `<th>` is not focusable or announced as interactive. A button gets keyboard focus, Enter/Space activation and the right role for free. `aria-sort` on the `<th>` tells screen readers the current order.

Q18. Why use a real `<table>` and not divs with CSS grid?

Semantic tables give screen readers row/column navigation and header association out of the box. Divs need `role="table"`, `role="row"`, `role="cell"` and more to match. Use divs only when you need virtualization layouts the table element cannot do.

Q19. Why is the column config outside the component?

It is static. Defined inside the component, it would be a new array each render, which breaks the `useMemo` dependency on `columns` and re-runs filtering every time.

Performance

Q20. Where would this get slow and how do you profile it?

Rendering thousands of rows (DOM size) and sorting big arrays. Use React DevTools Profiler and the Performance tab. Fixes: paginate or virtualize rendering, memoize pipeline stages, `React.memo` on rows with stable callbacks, move work to the server or a Web Worker for very heavy sorts.

Q21. What if each page should show 5,000 rows?

Virtualize: render only the ~30 rows in the viewport plus a buffer, with spacer elements above and below (or a library like TanStack Virtual). Keep the sticky header and fixed row height to make the math simple.

Trade-offs

Q22. Why not use TanStack Table or AG Grid?

In production, often yes: they handle resizing, grouping, virtualization and edge cases. In an interview the goal is to show you understand the pipeline, state ownership and accessibility. I would mention the library and note that TanStack Table is headless, so it fits this same design.

Q23. How would you persist table state across refresh?

Read initial `q`, `sort`, `order`, `page`, `pageSize` from `URLSearchParams`, and write them back with `history.replaceState` when they change. The URL becomes the source of truth and the view is shareable.

Q24. How would you test this?

Unit-test the pure functions (sorting types, empty values, stability, pagination bounds). Component tests with React Testing Library: click a header and assert row order and `aria-sort`, type and wait for the debounce with fake timers, tick checkboxes and assert the indeterminate state. Mock `fetchUsers` to test loading, error and retry.